

Core Concepts — What Makes Something an Agent

Part 06 — 🧠 AI Agents · 06-ai-agents/notes/01-core-concepts.md

1. Agent vs. chatbot: the actual difference

A chatbot is a function: `text in -> text out`. You can wrap it in a nice UI, give it a system prompt, even let it use RAG — it is still one forward pass (or one streamed response) per user turn, and every action in the world is *you*, copying its answer somewhere.

An **agent** is a chatbot placed inside a loop that can act on its own outputs without you re-typing anything. Concretely, four things have to be true:

1. **Tools** — the model can cause side effects (call an API, run code, write a file, query a database) instead of only producing text for a human to read.
2. **A control loop** — the system re-invokes the model with the *result* of its last action, automatically, so the model can react to what just happened.
3. **Multiple steps toward a goal** — the loop runs more than once per user request, without a human approving each step in between.
4. **Some planning/memory framing** — the model tracks what it has already tried and what's left, even if "memory" is just the growing transcript in its own context.

None of these is exotic technology. A `for` loop, a `try/except`, and a system prompt that says "you have these tools" is a complete, if crude, agent. What made agents feel new in 2022-2023 was not new ML — it was realizing that an instruction-tuned LLM is a good enough *decision function* inside that loop to be worth building the loop around.

	Chatbot	Agent
Unit of work	one turn	a task, possibly many turns
Who acts on the world	the human, using the model's text	the model, via tools
Control flow	fixed: prompt -> response	a loop: observe -> think -> act -> observe
Failure surface	a bad reply	a bad reply plus every side effect it caused
When it stops	after one response	when <i>it</i> (or a guard) decides it's done — see §4

The framing to hold onto: *agent* = *model* + *tools* + *memory* + *planning*, wired through a **control loop**.

Every architecture in this module — ReAct, Plan-and-Execute, Reflexion, a LangGraph state machine — is a specific answer to "how do these four pieces talk to each other." None of them changes what the pieces are.

This module is about that loop for **one** agent. The moment you have several agents coordinating — a supervisor delegating to workers, peer agents negotiating a plan — you're in Module 07 — Agentic AI Systems [07-agentic-ai]. Read this module first; multi-agent systems are single-agent loops wired together, and every failure mode here (compounding error, tool misuse, runaway loops) gets *worse*, not better, with more agents.

2. Tool use and function calling: the mechanics, not the magic

Here is the sentence that dissolves most of the mystique: **a model does not "decide" to call a tool in any sense different from how it decides the next word of a sentence.** It is next-token prediction, over a vocabulary that happens to include tokens that, when parsed, mean "call this function with these arguments." That's it. The "decision" is a probability distribution shaped by training data full of examples where, after a user asks for something a tool can do, the highest-probability continuation is a tool-call block instead of prose.

This matters practically: a tool call can be wrong in exactly the ways any other generated text can be wrong — the model can pick a plausible-sounding but nonexistent function name (a hallucination, same mechanism as a hallucinated citation), it can generate arguments that are the wrong type, and it can do all of this *fluently and confidently*, because fluency and confidence are properties of the sampling process, not of the tool actually existing.

How the round trip actually works

1. You send: [system prompt] + [conversation] + [tool schemas as JSON]
2. Model emits either:
 - (a) normal text → done, show the user
 - (b) a tool-call block: {"name": "get_weather", "arguments": {"city": "Boston"}}
3. YOUR code parses (b), looks up the real function, executes it
4. You send the result back as a new message (role: "tool"), tagged with the call's id, appended to the conversation
5. Model is invoked again, now with the tool's output in context
6. Repeat from step 2 until the model emits (a)

Nothing in steps 3-5 involves the model. The model never runs code, never touches a network, never queries a database. It emits text that *you* choose to interpret as an instruction to do those things. This is worth internalizing because it reframes every security question in the field: the model is a text generator with excellent taste in which text to generate; the *agent* is the harness that turns that text into action, and every guarantee about safety, permissions and validation has to live in the harness, because the model provides none. See Module 13 — AI Security [13-ai-security] and §6 below.

The schema is the interface

Modern tool-calling APIs (OpenAI, Anthropic, Gemini) take a JSON Schema per tool:

```
{
  "name": "get_weather",
  "description": "Get current weather for a city. Use this whenever the user asks about weather,
temperature, or conditions in a specific place.",
  "input_schema": {
    "type": "object",
    "properties": {
      "city": {"type": "string", "description": "City name, e.g. 'Boston' or 'Paris'"}
    },
    "required": ["city"]
  }
}
```

The model is trained to produce arguments conforming to this schema, and most providers additionally constrain decoding so the JSON is *structurally* guaranteed valid (see Module 04 §2 [04-llm/notes/02-inference-and-adaptation.md] on constrained decoding). That kills one failure mode — syntactically broken JSON — but not two others: **wrong values inside valid JSON** (a city that doesn't exist, a negative quantity where zero was meant) and **wrong tool choice entirely** (calling `get_weather` when the user asked about weather *forecasts*, which is a different tool). Schema validity is necessary, not sufficient.

Parallel tool calls

Newer APIs let a single model turn emit *several* tool calls at once ("get weather in Boston AND Paris"), which your harness executes (concurrently, if independent) and returns as multiple tool-result messages before the next model call. This is a real throughput win — one round trip instead of N — but it multiplies the surface for the `compounding-error` math (see notes/02) if you don't validate every one of the returned results before feeding them all back.

Handling failure: the taxonomy that matters

Failure	What happened	What the harness must do
Hallucinated tool name	model emits a call to a function that isn't registered	catch by lookup miss; return an observation naming the <i>real</i> available tools — do not crash
Malformed arguments	JSON doesn't parse, or parses but fails schema validation	return the parse/validation error as an observation the model can read and correct, not a Python traceback
Wrong but valid arguments	schema-valid, semantically wrong (unknown city, out-of-range id)	let the <i>tool</i> raise a domain error with a clear message; catch it at the same boundary as above
Tool itself fails	the API is down, the query times out	distinguish retryable (timeout, 5xx) from terminal (404, permission denied) and say which in the observation

The unifying rule: **an agent can only recover from an error it can read**. A stack trace is optimized for a human debugging in an IDE; an observation is optimized for a model deciding what to try next. `code/react_loop_from_scratch.py` in this module implements exactly this taxonomy by hand — run it and watch a hallucinated tool name and a truncated-JSON argument both get caught and turned into recoverable text, not crashes. For how a real framework implements this same round trip with retries, validation and streaming built in, see `../16-ai-tech-stack/tracks/langchain/` [16-ai-tech-stack/tracks/langchain]; for how Claude Code's own tool-execution loop applies these exact principles (including permission prompts as a

human-in-the-loop recovery layer), see `../16-ai-tech-stack/tracks/claude-code/` [16-ai-tech-stack/tracks/claude-code].

3. ReAct: Reason + Act, in full

ReAct (Yao et al., 2022 — see papers/PAPERS.md [06-ai-agents/papers/PAPERS.md]) is the pattern underneath almost every tool-using agent you'll encounter, including the tool-calling APIs above, which are essentially ReAct with the parsing done for you by the provider.

The insight is simple but was not obvious in 2022: **reasoning-only** prompting (chain-of-thought) produces a plausible-sounding plan that can't check itself against reality — the model reasons its way to a hallucinated fact and states it with the same confidence as a true one. **Acting-only** prompting (just emit tool calls, no visible reasoning) is brittle when a step fails, because there is no articulated intent to revise — the model has no record of *why* it tried the thing that didn't work, so its next attempt is not obviously better than a retry. ReAct interleaves them: every action is preceded by a stated thought, and every thought after the first is conditioned on a real observation from the world, not just on the model's own prior reasoning. Reasoning grounds acting; acting grounds reasoning.

The prompt structure, literally

A classic ReAct prompt (few-shot exemplars, then the live task) looks like this:

```
Answer the following question as best you can. You have access to these tools:
```

```
calculator(expression: string) -- evaluate an arithmetic expression
search_kb(query: string) -- search the knowledge base
```

```
Use this format:
```

```
Question: the input question
Thought: reason about what to do next
Action: the tool to use, one of [calculator, search_kb]
Action Input: the input to the tool
Observation: the tool's result
... (Thought/Action/Action Input/Observation repeats as needed)
Thought: I now know the final answer
Final Answer: the final answer
```

```
Question: What is 15% of 240?
Thought: I should compute this with the calculator.
Action: calculator
Action Input: 240 * 0.15
Observation: 36.0
Thought: I now know the final answer
Final Answer: 15% of 240 is 36.
```

Modern tool-calling APIs replace the **Action:** / **Action Input:** text block with a structured tool-call object, and replace **Observation:** with a **role: "tool"** message — but the *shape* — thought (often now a hidden or "reasoning" pass), action, observation, repeat — is unchanged. If you understand the text-based ReAct prompt above, you understand what the function-calling API is doing under a nicer interface.

Why interleaving beats either alone (the actual argument)

- **Reasoning without acting** can't be checked. A chain-of-thought that says "the file is probably at `/etc/config.yml`" is indistinguishable, in its own text, from one that says "the file is at `/etc/config.yml`, confirmed by `ls`." Only the second is grounded, and grounding requires an action.
- **Acting without reasoning** can't explain itself or adapt. If a tool call fails, the next action has to be *some* function of the failure — but with no stated thought, there's no articulated hypothesis to revise, so the model tends to repeat the same action (this is a real observed failure mode, not a theoretical one — see §7 of `notes/02` for the numbers).
- **Interleaved**, each thought is grounded by the previous real observation, and each action is justified by a thought a human (or a log-reading engineer) can audit. This is also why ReAct transcripts are the natural unit of agent evaluation [`15-ai-evals`] — you can grade the *reasoning*, not just the final answer.

4. The control loop, precisely

Strip away the prompt formatting and every agent framework is this:

```
def agent_loop(task, tools, model, max_iterations=10):
    transcript = [system_prompt(tools), user_message(task)]
    for i in range(max_iterations):
        response = model.generate(transcript)          # THINK (+ maybe ACT, same call)
        transcript.append(response)

        if response.is_final_answer():                # a semantic exit
            return response.answer

        for call in response.tool_calls:              # ACT
            result = execute_tool_safely(call, tools)  # OBSERVE
            transcript.append(tool_result_message(call.id, result))

    return give_up_gracefully(transcript)             # the budget exit
```

Two things about this loop deserve more respect than they usually get:

Termination conditions. There are exactly two legitimate ways this loop ends, and one illegitimate one you must guard against:

1. **Semantic exit** — the model itself emits a final answer with no further tool calls. This is the *intended* exit, and it is entirely the model's judgment call.
2. **Budget exit** — a hard `max_iterations` (or a token budget, or a wall-clock timeout) fires first. This is *your* judgment call, made in advance, precisely because you cannot trust the model's judgment alone.
3. **Illegitimate: it never ends** — no semantic exit, no budget exit, an unbounded `while True`. This is not a hypothetical: a model stuck re-trying a failing tool call, or oscillating between two actions, will do so forever unless something external stops it. This is a **cost incident**, not just a bug — every iteration is a paid model call.

"The agent decides when it's done" is a real design problem, not a solved one. The model's stated confidence that it's finished has no calibrated relationship to whether the task actually succeeded — it can declare victory on a wrong answer exactly as fluently as on a right one (same root cause as hallucination, see Module 04 §9 [04-llm/notes/01-how-llms-are-built.md]). This is why production agents layer *external* checks on top of the model's self-report: a verifier step, a schema check on the final output, a human approval gate. LangGraph's answer to this same problem — a durable, resumable state machine with an explicit `recursion_limit` and support for a human-in-the-loop approval node — is the runtime-engineering side of this exact issue; read `../16-ai-tech-stack/tracks/langgraph/notes/01-core-concepts.md` §3 [16-ai-tech-stack/tracks/langgraph/notes/01-core-concepts.md] ("Cycles and how they end") right after this file — it is the same three-exit taxonomy above, written for a graph instead of a `for` loop.

`code/react_loop_from_scratch.py` in this module implements this exact loop with a real `max_iterations` guard that fires and *explains itself* — run it and read the message it prints when it gives up.

5. Memory: a taxonomy, not a vibe

"Memory" is used loosely enough in agent marketing to mean four genuinely different mechanisms. Confusing them is why teams build an expensive vector database for a problem that a bigger context window would have solved, or vice versa.

Kind	What it holds	Lifetime	How it's actually implemented
Working memory	the current task's transcript: messages, thoughts, tool results	one task/session	the model's context window , literally — nothing more exotic than the list of messages you send on every call
Episodic memory	records of <i>past</i> completed tasks/sessions	across sessions	a log or database of past transcripts, summarized or embedded, retrieved when relevant to a new task
Semantic memory	general facts and domain knowledge, not tied to any specific past episode	indefinite	a knowledge base or vector store queried by retrieval — this is exactly RAG [08-rag] doing double duty as an agent's long-term factual memory
Procedural memory	<i>how</i> to do things — learned strategies, successful tool-use sequences, corrected mistakes	indefinite, improves over time	the least standardized: distilled successful trajectories fed back as few-shot examples, a fine-tuned adapter, or a curated "playbook" file the agent consults and updates

Concrete implementation notes for each:

- **Working memory** is finite and expensive (see Module 04 §6-7 [04-llm/notes/01-how-llms-are-built.md] on context rot and lost-in-the-middle). An agent that just appends every tool result forever will exhaust it — large tool outputs (a full file, a full API response) need summarization or truncation *before* they enter the transcript, not after. This is failure mode §7 in `notes/02`.
- **Episodic memory** is usually cheap to add and easy to get wrong: storing raw transcripts is easy, but retrieving the *right* past episode for a new task is a retrieval-quality problem, identical in kind to RAG

chunk retrieval — same embedding choices, same relevance-ranking failure modes.

- **Semantic memory** is where most production agents should spend their engineering budget before touching fine-tuning, for the same reason argued in Module 04 §3 [04-llm/notes/02-inference-and-adaptation.md]: retrieval supplies current, auditable facts; the model's weights should not be where you store "knowledge that changes."
- **Procedural memory** is genuinely immature as a discipline — most systems fake it with a growing "lessons learned" text file injected into the system prompt, which works until that file itself causes context rot. Reflexion-style self-critique (notes/02 §1) is the closest thing to a principled version: a critique of *this* trajectory becomes a memory consulted on the *next* one.

6. Where this returns

Idea here	Where it comes back
Planning strategies, when planning helps vs. hurts	notes/02-planning-failure-and-evals.md [06-ai-agents/notes/02-planning-failure-and-evals.md]
Tool design as a discipline, compounding error math	same file, §2-3
The ReAct loop as a runtime, durable and resumable	../16-ai-tech-stack/tracks/langgraph/
Tool-calling implementation details (LangChain)	../16-ai-tech-stack/tracks/langchain/
Semantic memory / retrieval at scale	Module 08 — RAG [08-rag]
Multiple agents coordinating	Module 07 — Agentic AI Systems [07-agentic-ai]
Untrusted tool output reaching the model — the lethal trifecta	notes/02 §5 [06-ai-agents/notes/02-planning-failure-and-evals.md], Module 13 — AI Security [13-ai-security]
Grading a trajectory, not just an answer	Module 15 — AI Evaluation [15-ai-evals]